AEGISPAY

Architecture V3 — Production Trust & Growth Layer for Agentic Commerce

AI proposes. AegisPay deterministically validates, authorizes and controls. Razorpay executes only approved financial actions.

1. What is AegisPay?

AegisPay sits between AI agents and payment providers. It does three jobs, and PROTECT is built in — so the AI is useful without ever being trusted with money. This document is the production design.

Job	What it means
GROW	AI helps a merchant increase revenue via upsell, cross-sell, bundles, campaigns and experimentation.
SELL	AI buyers understand intent, discover products, recommend, build carts, get authorization and pay through Razorpay.
PROTECT	AI can reason and recommend, but never directly controls money.

2. GROW / SELL / PROTECT

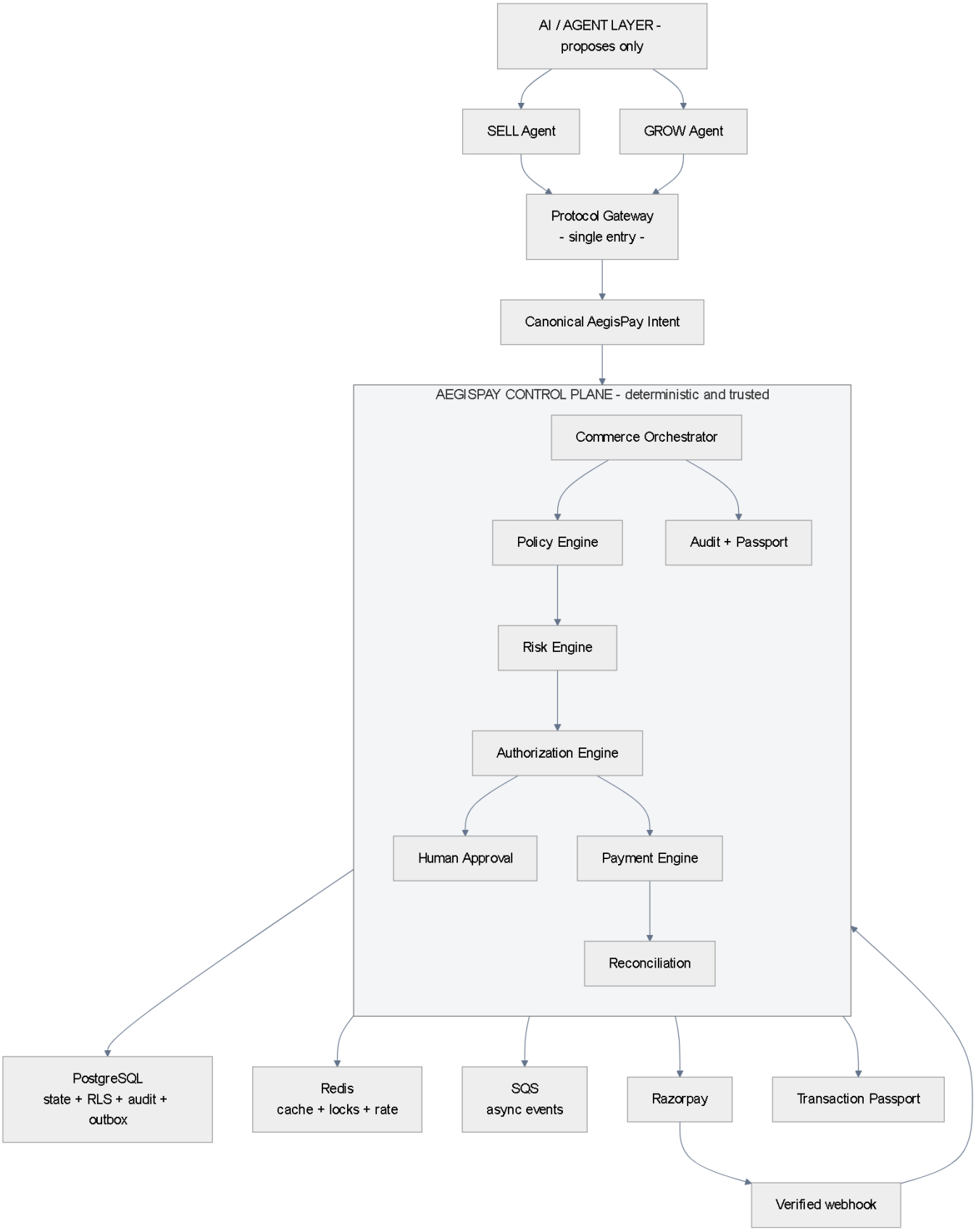


Figure 1 — The full system: AI layer (top), AegisPay control plane (middle), data + Razorpay (bottom)

Read it top to bottom: agents propose a structured intent; the control plane runs policy, risk and authorization; only then does it call Razorpay. Every step is recorded.

3. Architecture overview

Two focused services, one control plane owning all financial state, and boring, reliable infrastructure.

Layer	Component	Owns
AI / Agent	SELL Agent, GROW Agent	proposals, recommendations, structured intents
Control Plane	Commerce Orchestrator, Policy, Risk, Authorization, Payment, Approval, Reconciliation, Audit	financial state, determinism, provider interaction, money path
Data	PostgreSQL (state, RLS, audit, outbox), Redis (cache/locks/rate), SQS (async)	truth, isolation, reliability
External	Razorpay, verified webhooks	money movement, provider truth

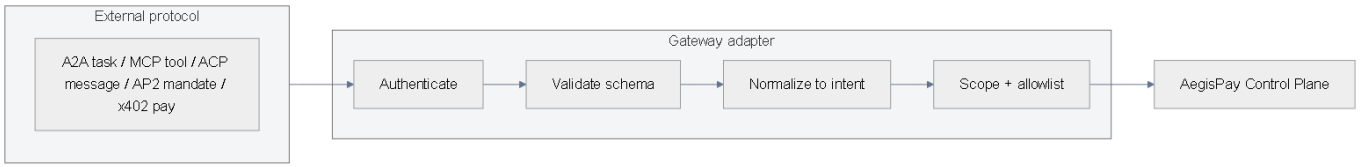
4. Component architecture

Component	What it does	Fails safely by
Commerce Orchestrator	Runs the purchase journey	Only allowed state transitions
Policy Engine	Fixed limits (amount, category, hours)	Denies if unsure
Risk Engine	Explainable score + why	Escalates to a human on doubt
Authorization Engine	Binds consent to one transaction	Expires, single-use
Payment Engine	Talks to Razorpay via one adapter	Returns UNKNOWN, never guesses
Human Approval	Person decides high value	Scoped, expiring, non-reusable
Webhook Gateway	Verifies + dedupes provider events	Drops bad/duplicate events
Reconciliation	Resolves UNKNOWN from provider truth	Never retries blindly
Audit + Passport	Records every decision	Tamper-evident chain

4b. The Protocol Gateway & agentic-commerce protocols

Every external protocol enters through **one** Protocol Gateway and is normalized into a single canonical AegisPay intent before the control plane. A new protocol is a new adapter — it changes the transport, never the money path.

Gateway — all protocols collapse into one canonical intent



Each adapter authenticates, validates, normalizes and scope-checks before the control plane

4c. Protocol maturity & security

Protocol	Role in AegisPay	Maturity
MCP	controlled agent tools / context	Core
A2A	agent-to-agent communication	Adapter-ready
UCP	commerce interoperability	Adapter-ready
ACP	agentic checkout / commerce	Adapter-ready
AP2	payment mandates & verifiable authorization	Experimental
x402	machine / pay-per-use payments	Experimental
A2UI	agent-driven UI	Experimental
NPCI UAP / UPI	India agentic-payment ecosystem	Future

Maturity is honest: **Core** is implemented, **Adapter-ready** has a mapping, **Experimental** maps only where semantics align, **Future** is a watch-list with no compliance claim. The core never imports a protocol SDK.

Protocol security	How it's handled
Authentication / identity	OAuth per protocol mapped to a canonical agent_id
Schema validation	strict typed schema on the normalized intent; free text never trusted
Replay protection	single-use nonce, expiry, binding to a transaction digest
Scoped authorization + idempotency	one action, one amount, one policy version; key per command
Tool allowlisting + tenant isolation	only safe tools exposed; tenant resolved from auth, RLS at the DB

5. AI / Control-plane trust boundary

Figure 2 — The AI thinks; the control plane decides and acts, sharing only a structured intent

The AI runtime has **no payment keys, no database credentials, no unrestricted money tools and no policy mutation capability**. A manipulated agent can therefore only produce an intent that the control plane will deterministically gate.

6. Service boundaries

Exactly two services. The AI runtime is isolated by privilege, not by language — both are FastAPI, but only the control plane can touch money.

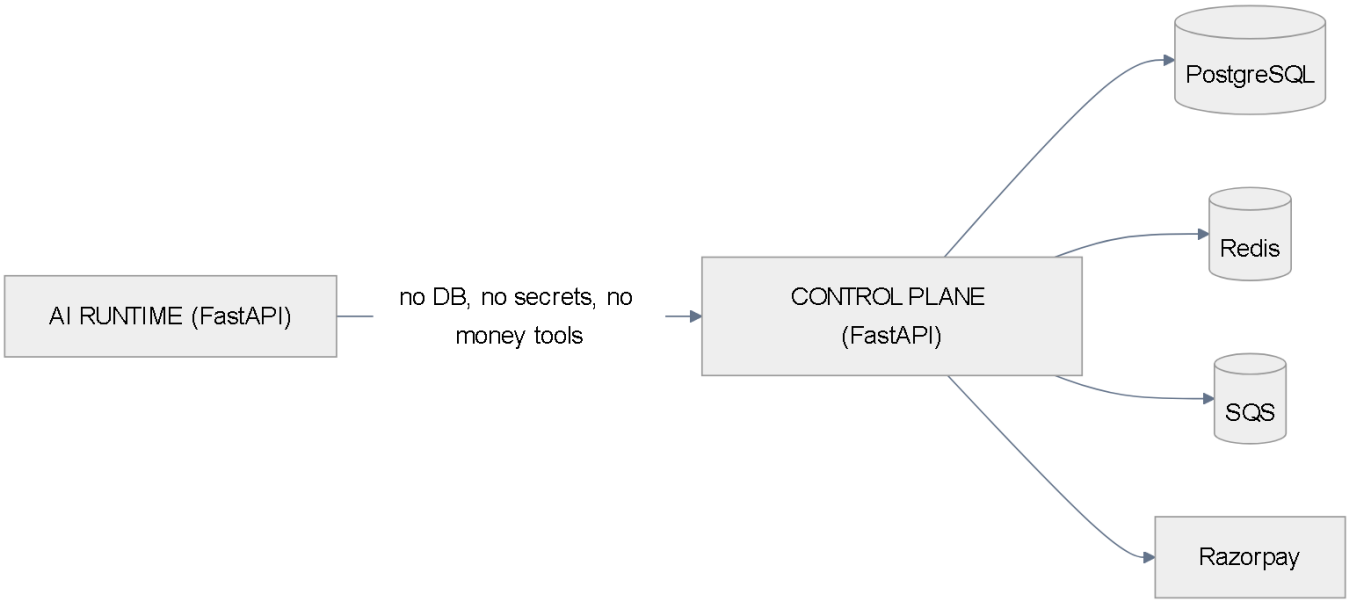


Figure 3 — AI Runtime (no DB, no secrets, no money tools) and Control Plane (owns state, policy, authz, provider, audit)

7. SELL flow

Intent → catalog → recommendation → cart → authorization → Razorpay → verified webhook → reconciliation → Passport. The full walkthrough is in the SELL document.

8. GROW flow

Data → opportunity → proposal → rules → estimate → approval → budget → execution → A/B → incremental measurement → learn. Full detail in the GROW document.

9. Policy / Risk / Authorization

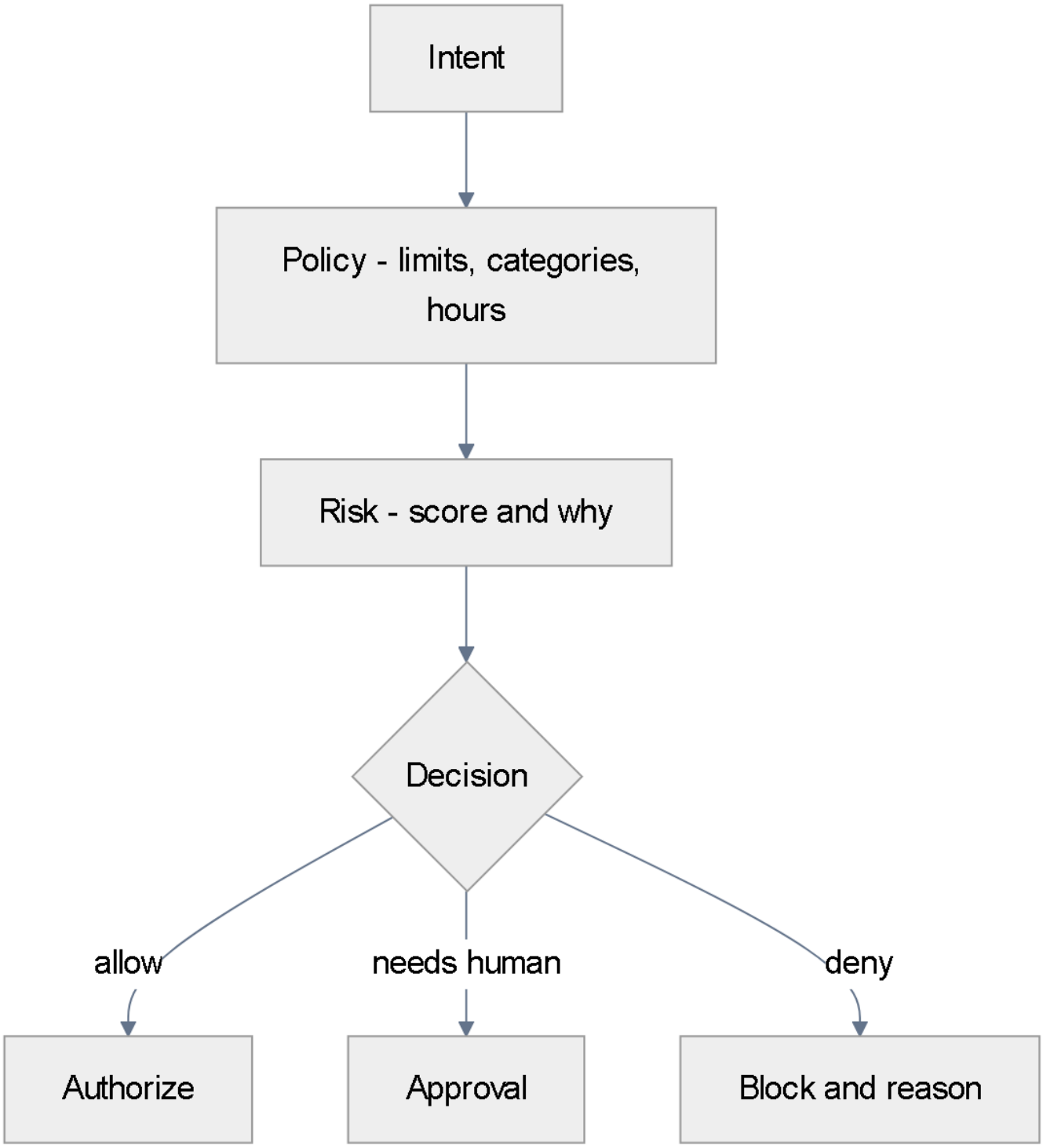


Figure 4 — Every money action passes a deterministic gate

• **Policy** is versioned and merchant-owned; the AI cannot change it.

• **Risk** is explainable: a score plus the reasons.

• **Authorization** is scoped, transaction-bound, expiring and single-use.

10. Human approval

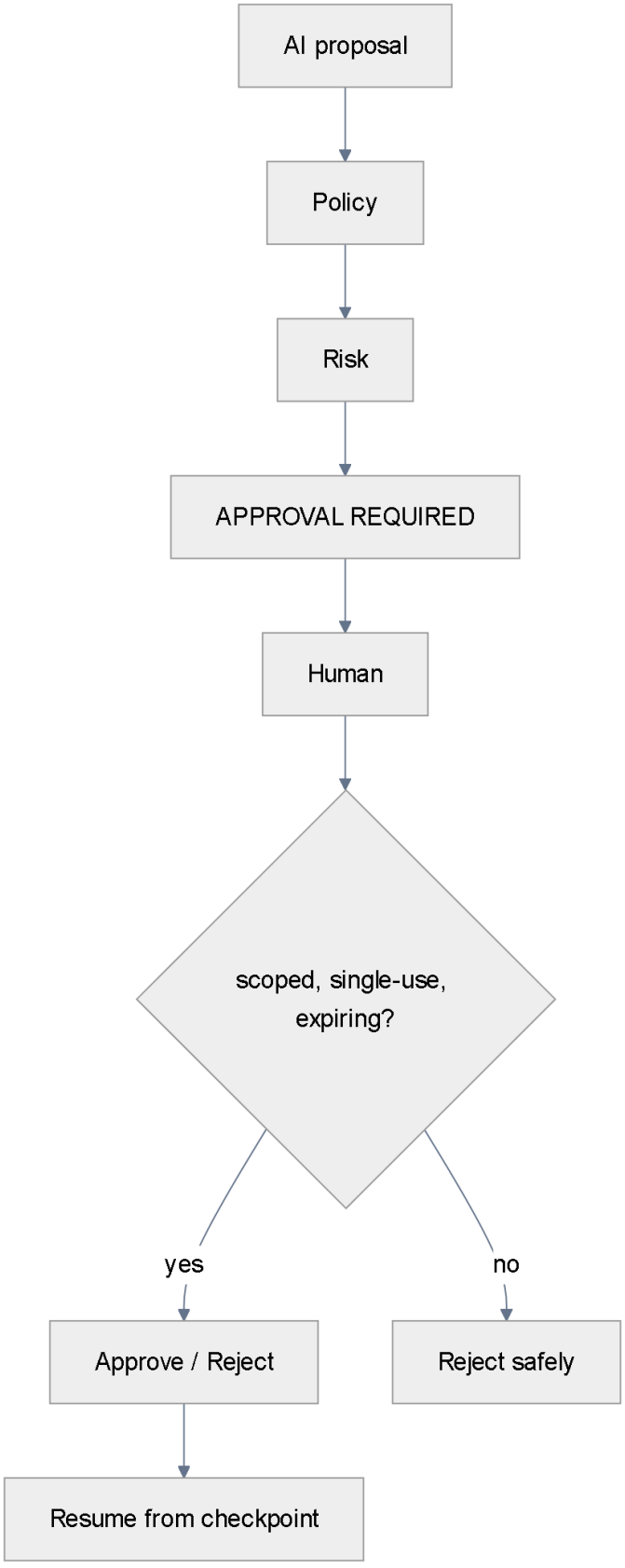


Figure 5 — A proposal escalates to a person only when required, and resumes from a checkpoint

Approval is scoped to one action, single-use, transaction-bound, expiring and immutable after issuance — it cannot be replayed or stretched into a bigger purchase.

11. Payment state machine

The full lifecycle, with failure/recovery states. **UNKNOWN is a first-class state** and exits only on provider truth. We never blindly retry an UNKNOWN payment.

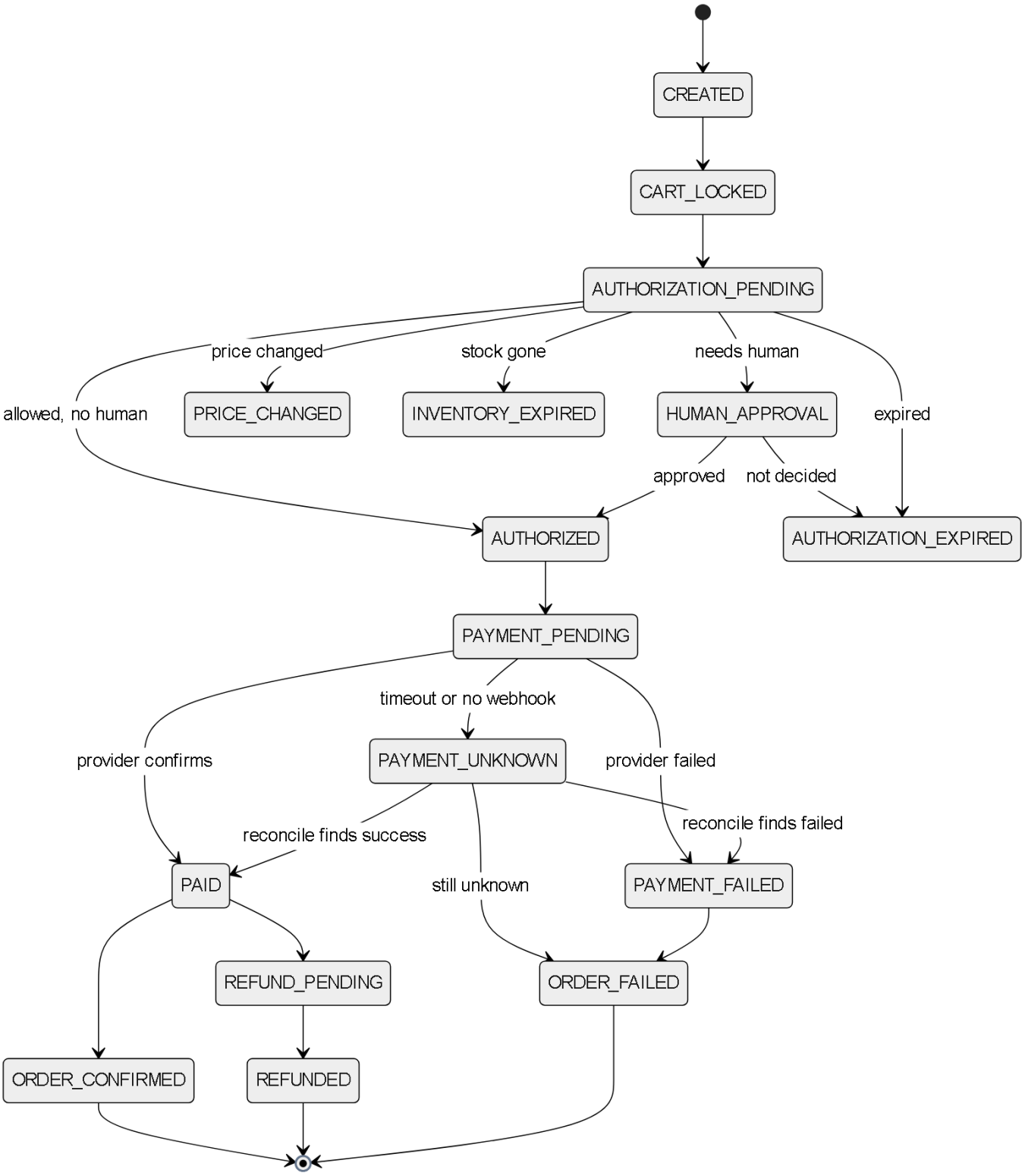


Figure 6 — Complete payment state machine, including failure and recovery states

12. Webhook + idempotency

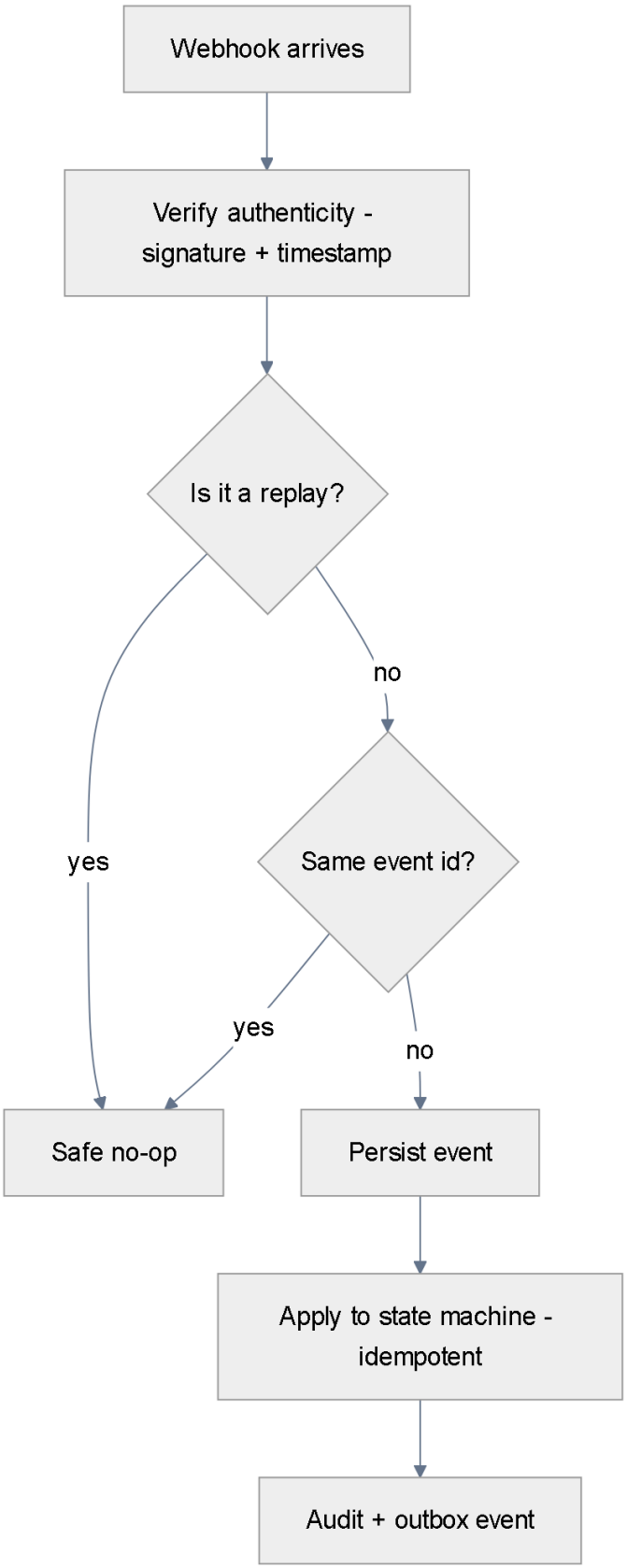


Figure 7 — Webhooks are untrusted until verified and deduped; duplicates are safe no-ops

Command	Idempotency key scope	Effect
Create order	order key + request hash	same request returns the same order
Payment operation	payment key	retry returns the prior provider result

Command	Idempotency key scope	Effect
Refund	(payment, refund key)	one effective refund per key
Webhook processing	(provider, event id)	duplicates are safe no-ops
Campaign budget	(campaign, spend request)	atomic reservation, never double-spends

Every financial command carries an idempotency key. This is **designed to prevent duplicate financial effects** — we do not claim duplicate payments are impossible.

13. Transactional Outbox

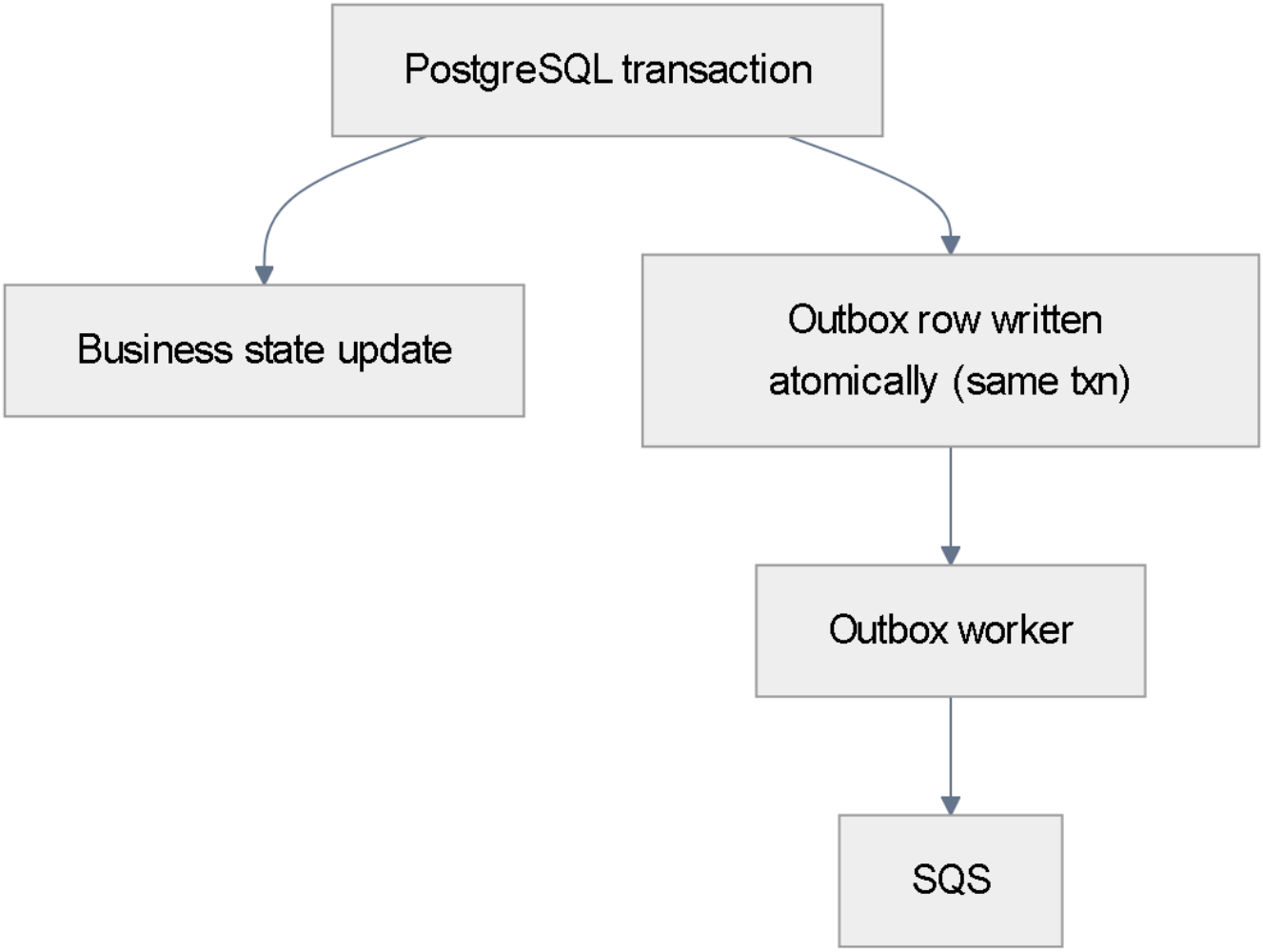


Figure 8 — A state change and its event are written in one transaction

Because the business-state update and the outbox event commit together, you cannot get a state change with no event (committed but not emitted) or an event with no state change (emitted but not committed). The outbox worker then publishes to SQS at-least-once, and consumers are idempotent.

14. Reconciliation

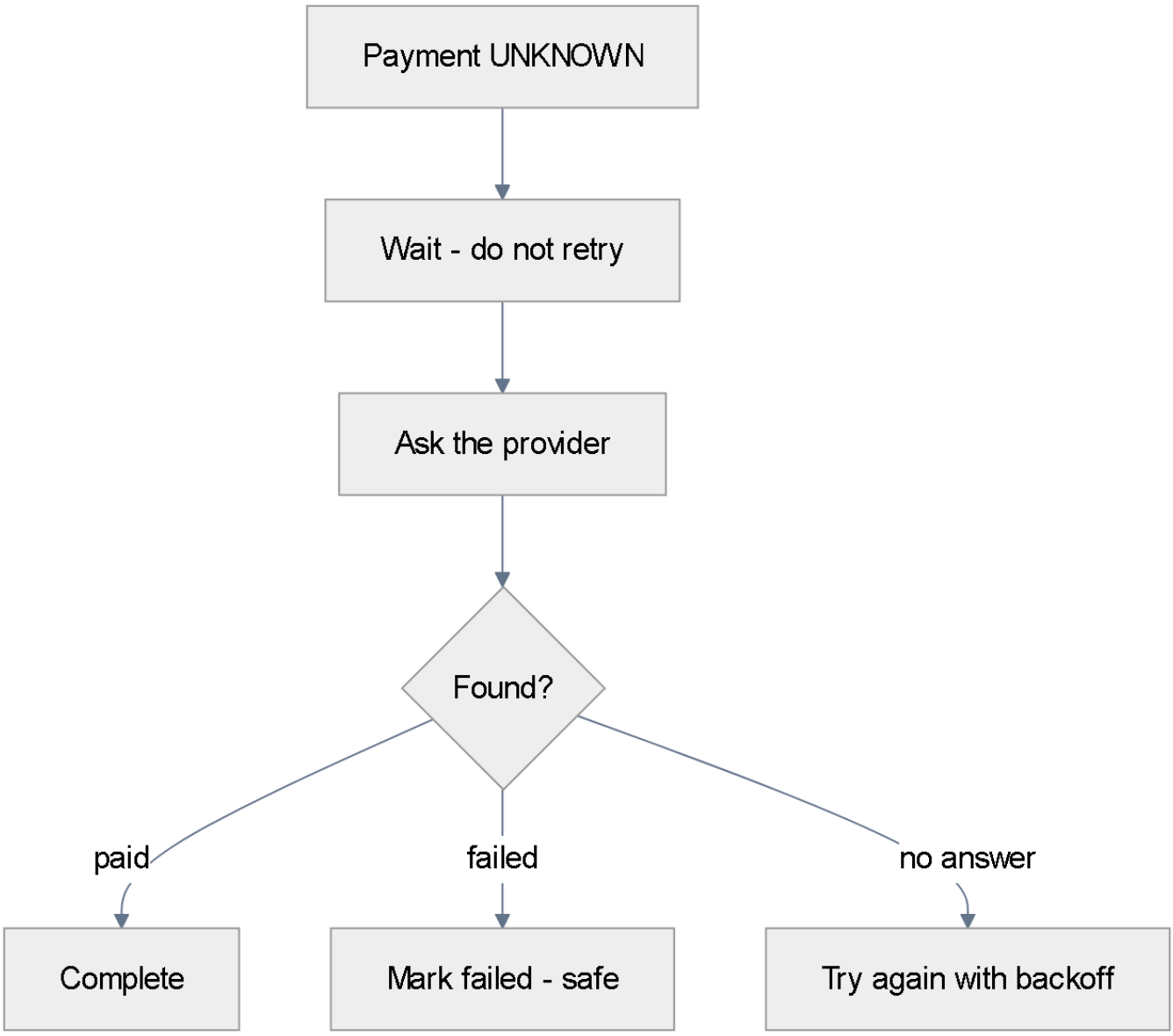


Figure 9 — UNKNOWN payments are checked, never blindly retried

A timed-out or unknown payment is never re-created. We wait, ask the provider, then complete or fail safely, with backoff and escalation. This is designed to prevent duplicate financial effects.

15. Cart / Price / Inventory protection

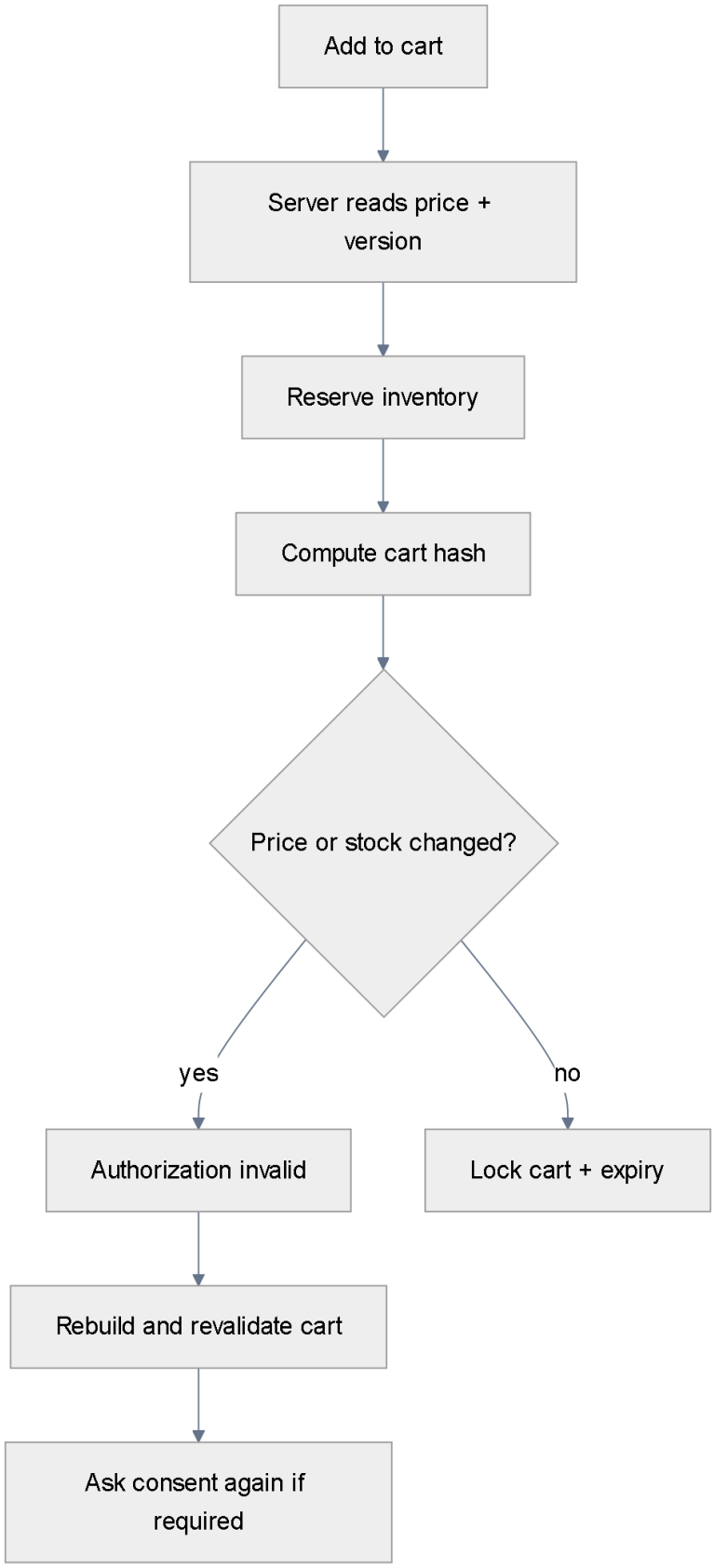


Figure 10 — If price, version or stock changes, authorization becomes invalid

Guard	What it does
Server-owned prices	Price comes from the DB, never the AI or client
Cart hash	Snapshot; a change invalidates authorization

Guard	What it does
Price version	A price change is detected and checkout re-validated
Inventory reservation	Stock is held during checkout so it can't oversell
Cart expiry	A stale cart cannot be paid
Product ownership lock	A product belongs to exactly one merchant (tenant). SKU is unique per merchant, and a reference must resolve to the same merchant as the cart — two merchants cannot share or add the same product, and a cross-merchant product in a cart is rejected.

Because products are tenant-owned and referenced only within their own merchant, a product from another merchant can never be injected into a purchase. This is enforced by the tenant context on every query plus a `unique (tenant_id, sku)` constraint.

16. Refund architecture

Refunds are controlled. The AI has **no** unrestricted refund tool — a refund needs policy, authorization and Razorpay verification.

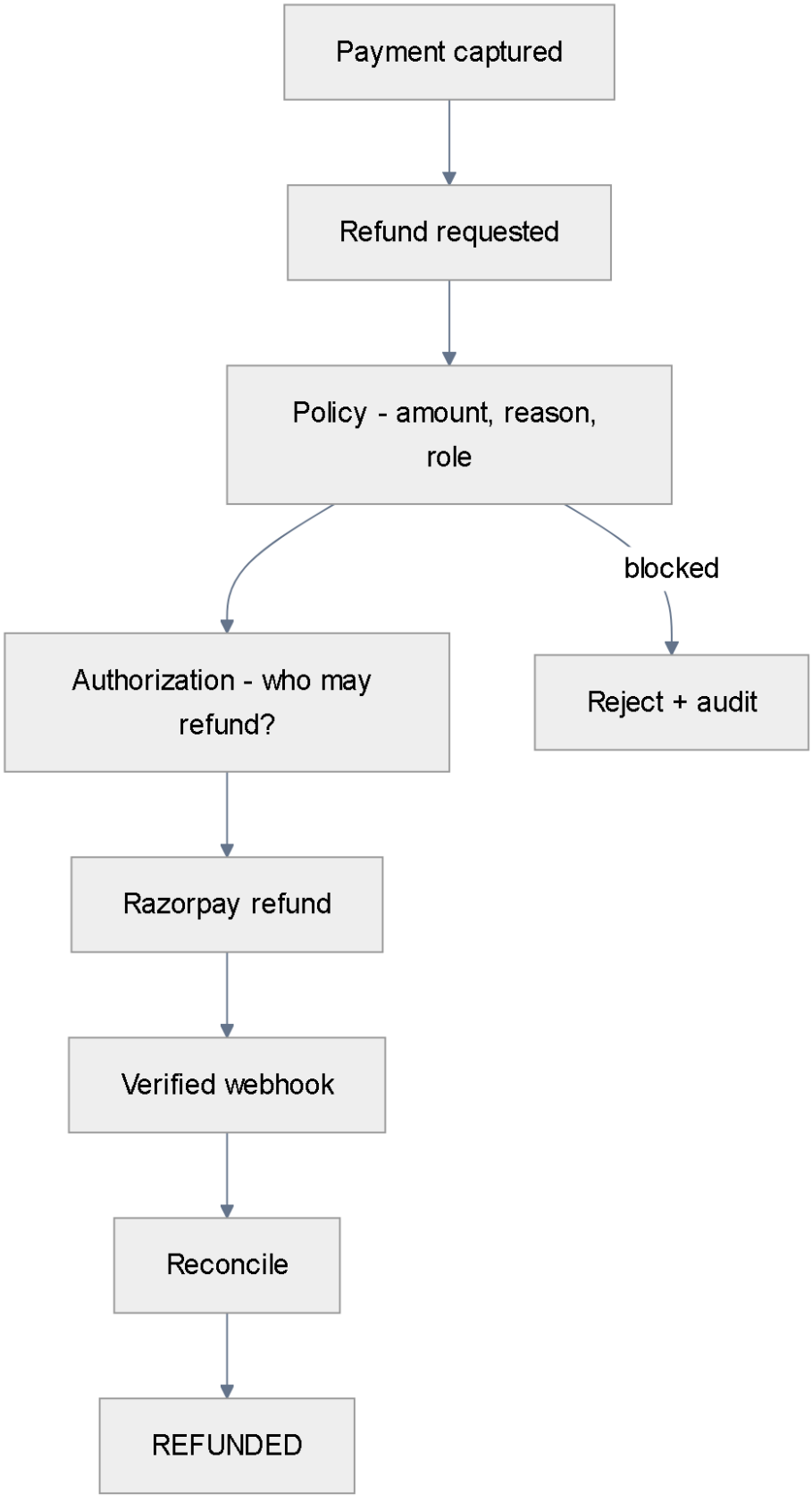


Figure 11 — Refund flow gated by policy and authorization

17. Agent security

Threat	Prevention
Prompt injection	Catalog is data, not instructions; intent schema-validated
Tool abuse	Only safe tools exposed; no money tool

Threat	Prevention
Agent impersonation	Signed credentials, session binding, revocation
Credential theft	Hashed, rotatable, scoped keys
Authorization replay	Transaction-bound, single-use, expiring authz
Self-escalation	No tool to change own limits/policy
Malicious catalog / merchant input	Typed, validated, injection classifier

18. Multi-tenancy + RLS

Shared PostgreSQL, shared schema, Row-Level Security. Two DB roles are kept separate so the application role cannot bypass RLS.

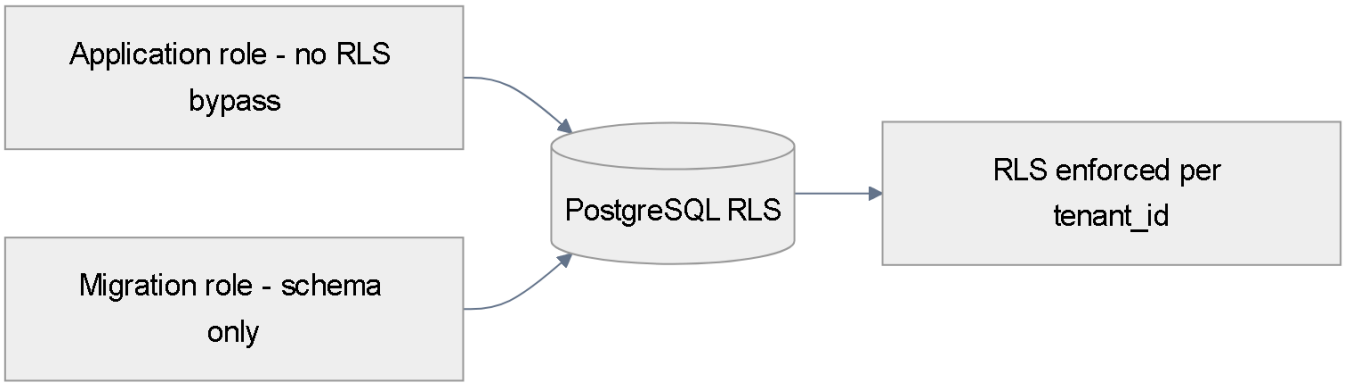


Figure 12 — Application role respects RLS; migration role only changes schema

• Application role: no DDL, no BYPASSRLS, scoped to tenant via a per-request `SET LOCAL app.tenant_id`.

• Migration role: used only during deploys for schema changes.

• Every request carries tenant context; tenant is derived from auth, never from the body.

• Products are tenant-owned: `unique (tenant_id, sku)` and a cart/order may only reference products of its own merchant. Two merchants cannot add or share the same product.

19. Audit + Transaction Passport

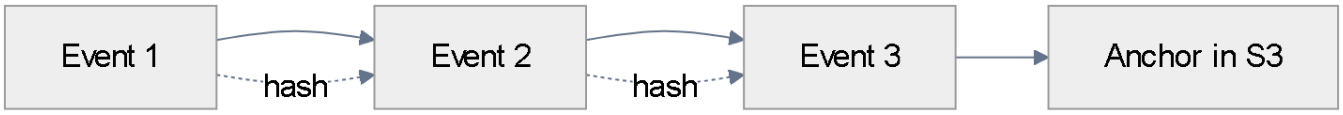


Figure 13 — Events are linked by hash into a tamper-evident, anchored chain

Append-only, RLS read-only, hash-chained and anchored to an immutable root. The Transaction Passport bundles the who/what/why/approval/provider so any purchase can be verified. It is **tamper-evident**, not tamper-proof.

20. AWS deployment

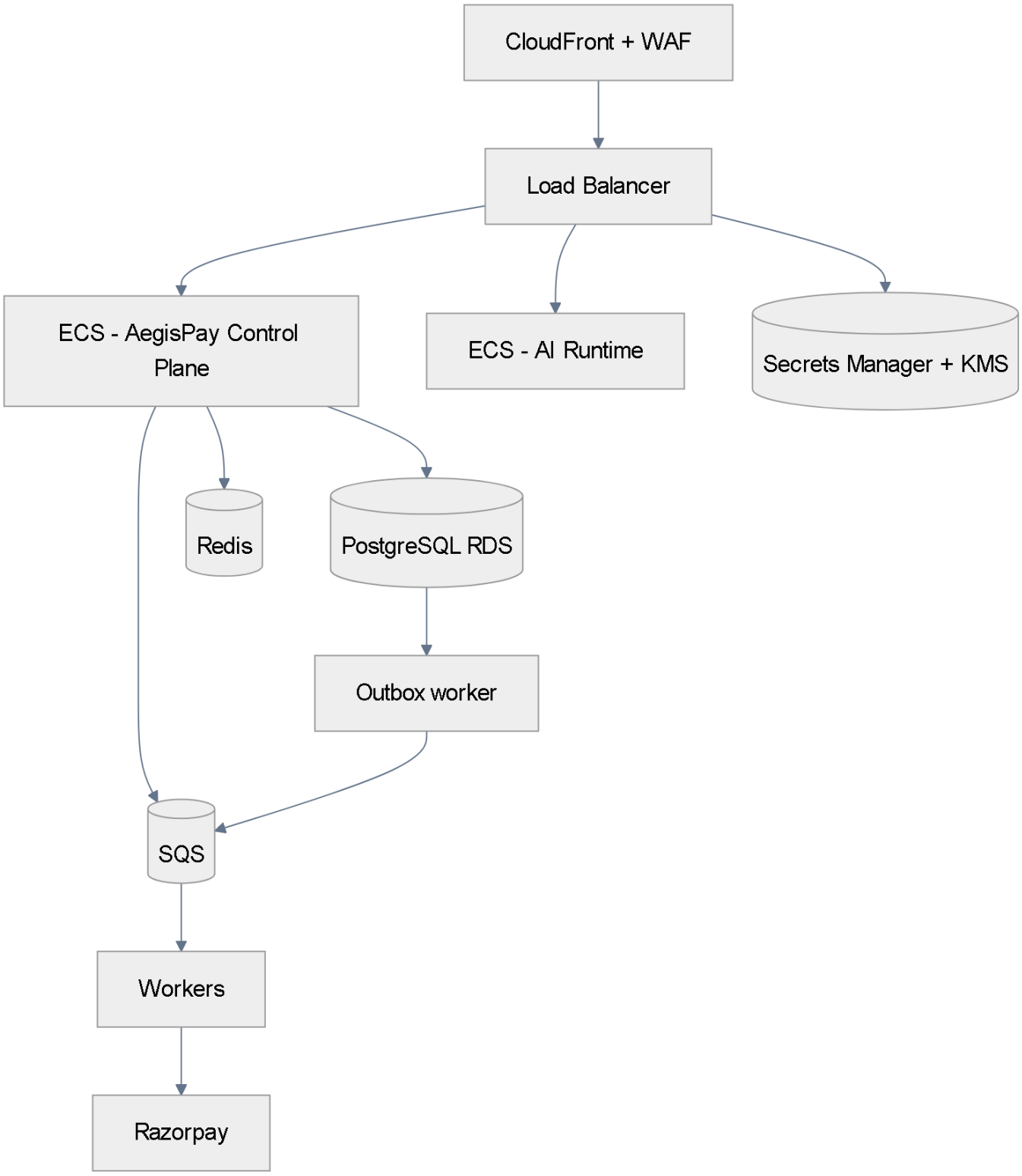


Figure 14 — Two ECS services, managed data, one queue

CloudFront + WAF → ALB → ECS (Control Plane + AI Runtime). PostgreSQL RDS, Redis, SQS, Secrets Manager + KMS. No Kubernetes — not needed at this scale.

21. Observability + SLOs

Dashboards cover the money path, the decision path and AI health. Every log carries correlation, tenant, transaction and agent ids; secrets and PII are never logged.

Signal	What it shows
Payment success / UNKNOWN rate	provider finality and ambiguity
Webhook latency / duplicate rate	ingest health and provider behaviour
Policy denial / risk escalation	decision guardrails
Human approval latency	time to a decision
Reconciliation backlog	how many are stuck UNKNOWN
AI tool failure / agent completion	agent reliability
Campaign ROI	growth effectiveness

SLOs (realistic, not fabricated): API availability ~99.9% (single-region multi-AZ, honest); RPO ≤ 15 min; RTO ≤ 1 hour; payment state transition typically < 2 s; webhook processing within seconds; reconciliation auto-resolves ≥ 95% of UNKNOWN; approvals expire on a timer.

22. Disaster recovery

Multi-AZ, PITR backups (RPO ≤ 15 min), quarterly restore drills, documented RTO ≤ 1 hour. Audit integrity is independent of DB restore via the hash chain + S3 anchor.

23. Threat model

Threat	Attack surface	Impact	Prevent	Detect	Recover
Prompt injection	catalog, prompts	unauthorized spend	catalog DATA-only, schema, allowlist	injection + denial alerts	block, revoke, audit
Tool abuse	agent tool calls	misuse	allowlist, rate limit, budget	anomaly alerts	throttle, suspend
Agent impersonation	credentials	spend as another agent	signed creds, mTLS, session binding	auth anomalies	revoke, re-issue
Webhook spoofing	webhook endpoint	false state	signature + timestamp verify	signature failure alerts	ignore, reconcile
Replay attack	authz / webhook / approval	repeat an action	single-use authz, dedupe, nonce	replay detectors	reject
Duplicate payment	retries	double effect	idempotency keys	duplicate detectors	return prior result
Price manipulation	cart / intent	wrong amount	server-owned prices + hash	price mismatch	reject
Tenant escape	queries	cross-tenant leak	RLS + tenant context + app role	isolation tests	block + audit
Authorization replay	authz reuse	second spend	scoped, expiring, single-use	duplicate authz	deny, revoke
Compromised AI runtime	LLM service	attempt money path	no secrets, no money tools, allowlist	red-team + anomalies	quarantine

24. Failure testing

Every failure behaves safely: detect → safe state → recover → audit. Introduced via a chaos/red-team harness in staging.

Fault	Detection	Safe state	Recovery	Audit
Razorpay timeout	long call / missing webhook	PAYMENT_UNKNOWN	reconcile → complete/fail	payment.unknown, payment.reconciled
Duplicate webhook	same event id	no-op	acknowledge only	webhook.deduped
Invalid webhook	signature check	reject, no state	alert + ignore	webhook.failed
Price changed in checkout	price version mismatch	authorization invalid	revalidate, ask consent	cart.rejected
Inventory unavailable	reservation fails	cart not locked	re-select / notify	inventory.reserve_failed
AI unavailable	timeout	agent path degrades	control plane still reconciles	agent.unavailable
Policy violation	policy decision	DENY, no payment	record reason	policy.denied
Expired approval	approval timer	expire, no action	re-request approval	approval.expired
DB / outbox failure	publish error	transaction rolls back	retry from outbox	outbox.retry
Campaign budget exhaustion	atomic check	deny / pause	no spend	campaign.budget_exceeded

25. Production readiness checklist

Area	Present
AI	No money path, tool allowlist, structured output, policy + risk, human approval, red-team
Payments	Idempotency, full state machine, webhook verify + dedupe, reconciliation, refund caps
Trust	Versioned policy, explainable risk, scoped/expiring authz, Transaction Passport, outbox
Growth	Atomic budget ledger, kill switch, A/B + incremental measurement
Security	Auth (user, agent, service), secrets isolation, RLS app role, encryption, rate limits
Reliability	Timeouts, retries, circuit breakers, outbox + DLQ, backups + DR, SLOs
Observability	Metrics, logs with IDs, tracing, alerts, dashboards

Why AegisPay is different

AI can reason and recommend, but only the deterministic AegisPay control plane validates, authorizes and controls financial actions. Razorpay executes only approved actions.

• **No direct money path.** The AI runtime has no payment keys, no DB credentials, no unrestricted money tools.

• **Deterministic authorization.** Policy, risk and a scoped, expiring authorization gate every action.

• **Provider truth.** Payment finality comes only from a verified webhook or reconciliation — never a guess.

• **Prevents, not promises.** Idempotency + UNKNOWN-never-blind-retry are designed to prevent duplicate financial effects.

• **Tamper-evident.** Every decision is recorded in a hash-chained, anchored audit ledger and Transaction Passport.

• **Simple to run.** Two services, PostgreSQL + Redis + SQS on ECS — no unnecessary complexity.